

*****Trans Fraud Detection Risk Analysis*****



Introduction

This project analyzes **1 million financial transactions across 50,000 accounts** to identify fraudulent transaction patterns, high-risk customer behavior, security vulnerabilities, and potential fraud networks.

The analysis combines transaction-level data with account profiles, fraud-pattern statistics, network relationships, and time-based transaction behavior. Using **Python, PostgreSQL, SQL, and Power BI**, the objective was to transform raw transaction data into actionable insights that can help a financial institution detect suspicious activity earlier, prioritize high-risk accounts, and reduce potential fraud losses.

Problem Statement

Financial institutions process a large volume of transactions every day, making it difficult to identify fraudulent activity using transaction amount alone. Fraud can be associated with unusual transaction behavior, unknown devices, foreign transactions, high transaction velocity, weak authentication, suspicious accounts, and coordinated networks of accounts.

The business needs to understand **where fraud is occurring, which behaviors are strongly associated with fraud, which accounts and transaction patterns require immediate attention, and where fraud prevention controls can be strengthened.**

The project therefore focuses on identifying the major drivers of fraud and translating them into practical actions for **fraud detection, account monitoring, authentication controls, and loss reduction.**

Goal

The goal of this project is to:

Quantify the overall **fraud volume, fraud rate, and financial loss.**

Identify transaction behaviors strongly associated with fraudulent activity.

Detect **high-risk accounts and suspicious transactions.**

Identify the most significant **fraud patterns and merchant categories**.

Analyze temporal patterns to determine when fraud risk is highest.

Identify suspicious relationships and potential **fraud rings**.

Provide business recommendations that can help reduce fraud exposure and improve transaction monitoring.

```
In [3]: import pandas as pd
import numpy as np

# Folder path
folder = r"C:\Users\HP\Downloads\archive (3)"

# File names (without .csv)
files = [
    "transactions",
    "account_profiles",
    "fraud_patterns",
    "network_edges",
    "time_series_stats"    # Use underscore if that's the actual file name
]

# Load all CSV files into a dictionary
tables = {
    file: pd.read_csv(fr"{folder}\{file}.csv")
    for file in files
}

# Access individual DataFrames
transactions = tables["transactions"]
account_profiles = tables["account_profiles"]
fraud_patterns = tables["fraud_patterns"]
network_edges = tables["network_edges"]
time_series_stats = tables["time_series_stats"]
```

```
In [4]: for name, df in tables.items():
    print("=" * 60)
    print(f"Table: {name}")
    print("=" * 60)
    print("Shape:", df.shape)
    print("\nColumns:")
    print(df.columns.tolist())
    print("\nFirst 5 rows:")
    print(df.head())
    print("\nMissing Values:")
    print(df.isnull().sum())
    print("\n")
```

=====

Table: transactions

=====

Shape: (1000000, 23)

Columns:

['transaction_id', 'account_id', 'timestamp', 'hour_of_day', 'day_of_week', 'is_weekend', 'amount', 'merchant_category', 'mcc_code', 'merchant_country', 'card_present', 'device_type', 'device_known', 'ip_risk_score', 'is_foreign_txn', 'time_since_last_s', 'velocity_1h', 'amount_vs_avg_ratio', 'account_age_days', 'has_2fa', 'credit_limit', 'is_fraud', 'fraud_pattern']

First 5 rows:

	transaction_id	account_id	timestamp	hour_of_day	day_of_week	\
0	TXN000000001	ACC0016173	2023-02-21 08:02:38	8	1	
1	TXN000000002	ACC0011196	2024-05-12 23:13:34	23	6	
2	TXN000000003	ACC0011181	2023-09-22 23:28:21	23	4	
3	TXN000000004	ACC0037105	2022-09-28 23:26:38	23	2	
4	TXN000000005	ACC0028471	2023-02-23 17:54:13	17	3	

	is_weekend	amount	merchant_category	mcc_code	merchant_country	...	\
0	0	168.42	travel	4511	CA	...	
1	1	85.78	online_retail	5999	AU	...	
2	0	20.15	pharmacy	5912	CA	...	
3	0	62.49	grocery	5411	US	...	
4	0	71.68	online_retail	5999	US	...	

	ip_risk_score	is_foreign_txn	time_since_last_s	velocity_1h	\
0	53.2	1	21	3	
1	25.3	1	234	1	
2	21.3	1	85	1	
3	13.7	0	98	0	
4	9.7	0	721	2	

	amount_vs_avg_ratio	account_age_days	has_2fa	credit_limit	is_fraud	\
0	2.6423	3256	1	3958.46	0	
1	0.7279	1527	1	3553.35	0	
2	0.1851	2230	1	4362.57	0	
3	1.5223	1863	1	3194.84	0	
4	0.7724	1728	0	11850.06	0	

	fraud_pattern
0	NaN
1	NaN
2	NaN
3	NaN
4	NaN

[5 rows x 23 columns]

Missing Values:

transaction_id	0
account_id	0
timestamp	0
hour_of_day	0
day_of_week	0
is_weekend	0
amount	0
merchant_category	0
mcc_code	0

```

merchant_country      0
card_present          0
device_type           0
device_known          0
ip_risk_score         0
is_foreign_txn        0
time_since_last_s     0
velocity_1h           0
amount_vs_avg_ratio   0
account_age_days      0
has_2fa               0
credit_limit          0
is_fraud              0
fraud_pattern         982857
dtype: int64

```

```

=====
Table: account_profiles
=====

```

```

Shape: (50000, 23)

```

```

Columns:

```

```

['account_id', 'account_age_days', 'credit_limit', 'home_country', 'risk_score',
 'is_high_risk', 'avg_txn_amount', 'avg_monthly_txns', 'has_2fa', 'account_type',
 'total_transactions', 'total_amount', 'avg_amount', 'max_amount', 'fraud_count',
 'fraud_amount', 'pct_foreign', 'avg_velocity', 'unique_countries', 'unique_catego
ries', 'avg_ip_risk', 'fraud_rate', 'is_fraudster']

```

```

First 5 rows:

```

	account_id	account_age_days	credit_limit	home_country	risk_score	\
0	ACC0000001	353	2171.42	US	16.5	
1	ACC0000002	2831	3031.38	US	25.4	
2	ACC0000003	2399	7533.75	US	10.6	
3	ACC0000004	1618	4821.94	US	20.7	
4	ACC0000005	1597	3355.10	US	30.6	

	is_high_risk	avg_txn_amount	avg_monthly_txns	has_2fa	account_type	...	\
0	0	90.82	71.1	1	personal	...	
1	0	63.78	7.4	1	business	...	
2	0	72.18	31.9	0	personal	...	
3	0	53.64	16.6	1	personal	...	
4	0	168.44	32.0	1	personal	...	

	max_amount	fraud_count	fraud_amount	pct_foreign	avg_velocity	\
0	740.46	0.0	0.00	0.35	1.27	
1	186.13	1.0	186.13	0.50	4.00	
2	1175.86	0.0	0.00	0.17	1.44	
3	2753.31	0.0	0.00	0.33	1.11	
4	718.61	0.0	0.00	0.42	0.95	

	unique_countries	unique_categories	avg_ip_risk	fraud_rate	is_fraudster
0	9.0	11.0	22.98	0.0	0.0
1	2.0	2.0	52.45	0.5	1.0
2	3.0	8.0	15.50	0.0	0.0
3	4.0	10.0	19.01	0.0	0.0
4	7.0	12.0	26.71	0.0	0.0

```

[5 rows x 23 columns]

```

Missing Values:

account_id	0
account_age_days	0
credit_limit	0
home_country	0
risk_score	0
is_high_risk	0
avg_txn_amount	0
avg_monthly_txns	0
has_2fa	0
account_type	0
total_transactions	0
total_amount	0
avg_amount	0
max_amount	0
fraud_count	0
fraud_amount	0
pct_foreign	0
avg_velocity	0
unique_countries	0
unique_categories	0
avg_ip_risk	0
fraud_rate	0
is_fraudster	0

dtype: int64

=====

Table: fraud_patterns

=====

Shape: (7, 12)

Columns:

['fraud_pattern', 'description', 'transaction_count', 'fraud_share_pct', 'avg_amount', 'median_amount', 'pct_night_0_5', 'pct_foreign', 'pct_card_not_present', 'avg_velocity_1h', 'avg_ip_risk', 'pct_no_2fa']

First 5 rows:

	fraud_pattern	description \
0	card_not_present	Online/CNP fraud – stolen card details used wi...
1	account_takeover	Fraudster gains access to legitimate account v...
2	card_present_stolen	Physical card stolen and used at POS terminals
3	friendly_fraud	Legitimate cardholder disputes valid transacti...
4	atm_fraud	ATM skimming or card trapping to clone card

	transaction_count	fraud_share_pct	avg_amount	median_amount \
0	5982	34.89	771.78	268.63
1	3432	20.02	1203.13	436.78
2	3120	18.20	556.84	200.27
3	1726	10.07	352.61	120.80
4	1216	7.09	466.26	164.53

	pct_night_0_5	pct_foreign	pct_card_not_present	avg_velocity_1h \
0	41.32	58.06	73.12	3.98
1	40.88	59.64	74.24	3.96
2	40.83	57.24	73.17	3.95
3	38.99	55.97	74.68	3.93
4	40.87	60.44	76.23	4.04

avg_ip_risk pct_no_2fa

0	55.26	42.24
1	55.89	41.72
2	54.99	40.61
3	54.91	42.47
4	55.60	39.39

Missing Values:

fraud_pattern	0
description	0
transaction_count	0
fraud_share_pct	0
avg_amount	0
median_amount	0
pct_night_0_5	0
pct_foreign	0
pct_card_not_present	0
avg_velocity_1h	0
avg_ip_risk	0
pct_no_2fa	0
dtype:	int64

Table: network_edges

Shape: (7411, 6)

Columns:

['account_a', 'account_b', 'shared_type', 'connection_count', 'ring_id', 'both_fraud']

First 5 rows:

	account_a	account_b	shared_type	connection_count	ring_id	\
0	ACC0017803	ACC0040032	phone	7	RING0001	
1	ACC0017803	ACC0042246	email_domain	12	RING0001	
2	ACC0017803	ACC0029491	phone	13	RING0001	
3	ACC0017803	ACC0022213	phone	12	RING0001	
4	ACC0017803	ACC0007601	ip_address	14	RING0001	

	both_fraud
0	1
1	1
2	1
3	1
4	1

Missing Values:

account_a	0
account_b	0
shared_type	0
connection_count	0
ring_id	3000
both_fraud	0
dtype:	int64

Table: time_series_stats

Shape: (26280, 10)

Columns:

['hour', 'transaction_count', 'fraud_count', 'total_amount', 'avg_amount', 'avg_ip_risk', 'fraud_rate', 'hour_of_day', 'day_of_week', 'is_weekend']

First 5 rows:

	hour	transaction_count	fraud_count	total_amount	\
0	2022-01-01 00:00:00	40	1	9341.66	
1	2022-01-01 01:00:00	43	0	6295.77	
2	2022-01-01 02:00:00	41	1	10763.95	
3	2022-01-01 03:00:00	34	0	9354.50	
4	2022-01-01 04:00:00	39	2	5897.09	

	avg_amount	avg_ip_risk	fraud_rate	hour_of_day	day_of_week	is_weekend
0	233.54	19.92	0.025000	0	5	1
1	146.41	20.32	0.000000	1	5	1
2	262.54	22.13	0.024390	2	5	1
3	275.13	22.00	0.000000	3	5	1
4	151.21	25.87	0.051282	4	5	1

Missing Values:

hour	0
transaction_count	0
fraud_count	0
total_amount	0
avg_amount	0
avg_ip_risk	0
fraud_rate	0
hour_of_day	0
day_of_week	0
is_weekend	0

dtype: int64

```
In [5]: network_edges[['ring_id']].tail(10)
```

```
Out[5]:
```

	ring_id
7401	NaN
7402	NaN
7403	NaN
7404	NaN
7405	NaN
7406	NaN
7407	NaN
7408	NaN
7409	NaN
7410	NaN

```
In [6]: network_edges['ring_id'] = network_edges['ring_id'].fillna('No Ring')
```

```
In [7]: network_edges.ring_id.isnull().sum()
```

```
Out[7]: np.int64(0)
```

```
In [8]: transactions['fraud_pattern'].value_counts()
```

```
Out[8]: fraud_pattern
card_not_present      5982
account_takeover      3432
card_present_stolen   3120
friendly_fraud        1726
atm_fraud             1216
money_laundering      1011
identity_theft         656
Name: count, dtype: int64
```

```
In [9]: transactions['fraud_pattern'] = transactions['fraud_pattern'].fillna('No Fraud')
```

```
In [10]: transactions['fraud_pattern'].isnull().sum()
```

```
Out[10]: np.int64(0)
```

```
In [11]: for name, df in tables.items():
          print("=" * 60)
          print(f"Table: {name}")
          print("=" * 60)
          print(df.info())
```



```
=====
Table: transactions
=====
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 23 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   transaction_id                        1000000 non-null object
1   account_id                           1000000 non-null object
2   timestamp                            1000000 non-null object
3   hour_of_day                          1000000 non-null int64
4   day_of_week                          1000000 non-null int64
5   is_weekend                           1000000 non-null int64
6   amount                               1000000 non-null float64
7   merchant_category                    1000000 non-null object
8   mcc_code                             1000000 non-null int64
9   merchant_country                     1000000 non-null object
10  card_present                         1000000 non-null int64
11  device_type                          1000000 non-null object
12  device_known                         1000000 non-null int64
13  ip_risk_score                        1000000 non-null float64
14  is_foreign_txn                       1000000 non-null int64
15  time_since_last_s                    1000000 non-null int64
16  velocity_1h                         1000000 non-null int64
17  amount_vs_avg_ratio                  1000000 non-null float64
18  account_age_days                     1000000 non-null int64
19  has_2fa                             1000000 non-null int64
20  credit_limit                         1000000 non-null float64
21  is_fraud                             1000000 non-null int64
22  fraud_pattern                        1000000 non-null object
dtypes: float64(4), int64(12), object(7)
memory usage: 175.5+ MB
None
=====
```

```
=====
Table: account_profiles
=====
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 23 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   account_id                           50000 non-null object
1   account_age_days                     50000 non-null int64
2   credit_limit                         50000 non-null float64
3   home_country                         50000 non-null object
4   risk_score                           50000 non-null float64
5   is_high_risk                         50000 non-null int64
6   avg_txn_amount                       50000 non-null float64
7   avg_monthly_txns                     50000 non-null float64
8   has_2fa                             50000 non-null int64
9   account_type                         50000 non-null object
10  total_transactions                    50000 non-null float64
11  total_amount                         50000 non-null float64
12  avg_amount                           50000 non-null float64
13  max_amount                           50000 non-null float64
14  fraud_count                          50000 non-null float64
15  fraud_amount                         50000 non-null float64
16  pct_foreign                          50000 non-null float64
17  avg_velocity                         50000 non-null float64
=====
```

```

18 unique_countries      50000 non-null float64
19 unique_categories      50000 non-null float64
20 avg_ip_risk            50000 non-null float64
21 fraud_rate             50000 non-null float64
22 is_fraudster           50000 non-null float64

```

dtypes: float64(17), int64(3), object(3)

memory usage: 8.8+ MB

None

Table: fraud_patterns

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 7 entries, 0 to 6

Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
0	fraud_pattern	7 non-null	object
1	description	7 non-null	object
2	transaction_count	7 non-null	int64
3	fraud_share_pct	7 non-null	float64
4	avg_amount	7 non-null	float64
5	median_amount	7 non-null	float64
6	pct_night_0_5	7 non-null	float64
7	pct_foreign	7 non-null	float64
8	pct_card_not_present	7 non-null	float64
9	avg_velocity_1h	7 non-null	float64
10	avg_ip_risk	7 non-null	float64
11	pct_no_2fa	7 non-null	float64

dtypes: float64(9), int64(1), object(2)

memory usage: 804.0+ bytes

None

Table: network_edges

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 7411 entries, 0 to 7410

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	account_a	7411 non-null	object
1	account_b	7411 non-null	object
2	shared_type	7411 non-null	object
3	connection_count	7411 non-null	int64
4	ring_id	7411 non-null	object
5	both_fraud	7411 non-null	int64

dtypes: int64(2), object(4)

memory usage: 347.5+ KB

None

Table: time_series_stats

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 26280 entries, 0 to 26279

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	hour	26280 non-null	object
1	transaction_count	26280 non-null	int64
2	fraud_count	26280 non-null	int64
3	total_amount	26280 non-null	float64

```

4  avg_amount      26280 non-null float64
5  avg_ip_risk     26280 non-null float64
6  fraud_rate      26280 non-null float64
7  hour_of_day     26280 non-null int64
8  day_of_week     26280 non-null int64
9  is_weekend      26280 non-null int64
dtypes: float64(4), int64(5), object(1)
memory usage: 2.0+ MB
None

```

```
In [12]: transactions['timestamp'] = pd.to_datetime(transactions['timestamp'])
```

```
In [13]: transactions.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 23 columns):
#   Column                Non-Null Count  Dtype
---  -
0   transaction_id         1000000 non-null object
1   account_id            1000000 non-null object
2   timestamp              1000000 non-null datetime64[ns]
3   hour_of_day            1000000 non-null int64
4   day_of_week            1000000 non-null int64
5   is_weekend             1000000 non-null int64
6   amount                1000000 non-null float64
7   merchant_category     1000000 non-null object
8   mcc_code               1000000 non-null int64
9   merchant_country      1000000 non-null object
10  card_present           1000000 non-null int64
11  device_type            1000000 non-null object
12  device_known           1000000 non-null int64
13  ip_risk_score          1000000 non-null float64
14  is_foreign_txn         1000000 non-null int64
15  time_since_last_s      1000000 non-null int64
16  velocity_1h            1000000 non-null int64
17  amount_vs_avg_ratio    1000000 non-null float64
18  account_age_days       1000000 non-null int64
19  has_2fa                1000000 non-null int64
20  credit_limit           1000000 non-null float64
21  is_fraud               1000000 non-null int64
22  fraud_pattern          1000000 non-null object
dtypes: datetime64[ns](1), float64(4), int64(12), object(6)
memory usage: 175.5+ MB

```

```
In [14]: transactions = transactions.rename(columns={
    'transaction_id': 'transaction_id',
    'account_id': 'account_id',
    'timestamp': 'transaction_datetime',
    'hour_of_day': 'transaction_hour',
    'day_of_week': 'transaction_day',
    'is_weekend': 'is_weekend',
    'amount': 'transaction_amount',
    'merchant_category': 'merchant_category',
    'mcc_code': 'merchant_category_code',
    'merchant_country': 'merchant_country',
    'card_present': 'card_present',
    'device_type': 'device_type',
    'device_known': 'device_known',
    'ip_risk_score': 'ip_risk_score',

```

```

    'is_foreign_txn': 'is_foreign_transaction',
    'time_since_last_s': 'seconds_since_last_transaction',
    'velocity_1h': 'transactions_last_1h',
    'amount_vs_avg_ratio': 'amount_vs_average_ratio',
    'account_age_days': 'account_age_days',
    'has_2fa': 'has_2fa',
    'credit_limit': 'credit_limit',
    'is_fraud': 'is_fraud',
    'fraud_pattern': 'fraud_pattern'
})

```

```

In [15]: account_profiles = account_profiles.rename(columns={
    'account_id': 'account_id',
    'account_age_days': 'account_age_days',
    'credit_limit': 'credit_limit',
    'home_country': 'home_country',
    'risk_score': 'account_risk_score',
    'is_high_risk': 'is_high_risk',
    'avg_txn_amount': 'average_transaction_amount',
    'avg_monthly_txns': 'average_monthly_transactions',
    'has_2fa': 'has_2fa',
    'account_type': 'account_type',
    'total_transactions': 'total_transactions',
    'total_amount': 'total_transaction_amount',
    'avg_amount': 'overall_average_amount',
    'max_amount': 'maximum_transaction_amount',
    'fraud_count': 'fraud_transaction_count',
    'fraud_amount': 'fraud_transaction_amount',
    'pct_foreign': 'foreign_transaction_pct',
    'avg_velocity': 'average_velocity',
    'unique_countries': 'unique_countries',
    'unique_categories': 'unique_categories',
    'avg_ip_risk': 'average_ip_risk',
    'fraud_rate': 'fraud_rate',
    'is_fraudster': 'is_fraudster'
})

```

```

In [16]: fraud_patterns = fraud_patterns.rename(columns={
    'fraud_pattern': 'fraud_pattern',
    'description': 'fraud_description',
    'transaction_count': 'fraud_transaction_count',
    'fraud_share_pct': 'fraud_share_pct',
    'avg_amount': 'average_transaction_amount',
    'median_amount': 'median_transaction_amount',
    'pct_night_0_5': 'night_transaction_pct',
    'pct_foreign': 'foreign_transaction_pct',
    'pct_card_not_present': 'card_not_present_pct',
    'avg_velocity_1h': 'average_transactions_last_1h',
    'avg_ip_risk': 'average_ip_risk',
    'pct_no_2fa': 'no_2fa_pct'
})

```

```

In [17]: network_edges = network_edges.rename(columns={
    'account_a': 'account_a',
    'account_b': 'account_b',
    'shared_type': 'connection_type',
    'connection_count': 'connection_count',
    'ring_id': 'fraud_ring_id',

```

```
    'both_fraud': 'both_accounts_fraud'
})
```

```
In [18]: time_series_stats = time_series_stats.rename(columns={
    'hour': 'transaction_datetime',
    'transaction_count': 'transaction_count',
    'fraud_count': 'fraud_count',
    'total_amount': 'total_transaction_amount',
    'avg_amount': 'average_transaction_amount',
    'avg_ip_risk': 'average_ip_risk',
    'fraud_rate': 'fraud_rate',
    'hour_of_day': 'transaction_hour',
    'day_of_week': 'transaction_day',
    'is_weekend': 'is_weekend'
})
```

```
In [19]: print("TRANSACTIONS")
print(transactions.columns.tolist())

print("\nACCOUNT PROFILES")
print(account_profiles.columns.tolist())

print("\nFRAUD PATTERNS")
print(fraud_patterns.columns.tolist())

print("\nNETWORK EDGES")
print(network_edges.columns.tolist())

print("\nTIME SERIES STATS")
print(time_series_stats.columns.tolist())
```

TRANSACTIONS

```
['transaction_id', 'account_id', 'transaction_datetime', 'transaction_hour', 'transaction_day', 'is_weekend', 'transaction_amount', 'merchant_category', 'merchant_category_code', 'merchant_country', 'card_present', 'device_type', 'device_known', 'ip_risk_score', 'is_foreign_transaction', 'seconds_since_last_transaction', 'transactions_last_1h', 'amount_vs_average_ratio', 'account_age_days', 'has_2fa', 'credit_limit', 'is_fraud', 'fraud_pattern']
```

ACCOUNT PROFILES

```
['account_id', 'account_age_days', 'credit_limit', 'home_country', 'account_risk_score', 'is_high_risk', 'average_transaction_amount', 'average_monthly_transactions', 'has_2fa', 'account_type', 'total_transactions', 'total_transaction_amount', 'overall_average_amount', 'maximum_transaction_amount', 'fraud_transaction_count', 'fraud_transaction_amount', 'foreign_transaction_pct', 'average_velocity', 'unique_countries', 'unique_categories', 'average_ip_risk', 'fraud_rate', 'is_fraudster']
```

FRAUD PATTERNS

```
['fraud_pattern', 'fraud_description', 'fraud_transaction_count', 'fraud_share_pct', 'average_transaction_amount', 'median_transaction_amount', 'night_transaction_pct', 'foreign_transaction_pct', 'card_not_present_pct', 'average_transactions_last_1h', 'average_ip_risk', 'no_2fa_pct']
```

NETWORK EDGES

```
['account_a', 'account_b', 'connection_type', 'connection_count', 'fraud_ring_id', 'both_accounts_fraud']
```

TIME SERIES STATS

```
['transaction_datetime', 'transaction_count', 'fraud_count', 'total_transaction_amount', 'average_transaction_amount', 'average_ip_risk', 'fraud_rate', 'transaction_hour', 'transaction_day', 'is_weekend']
```

KPI Calculation

```
In [20]: # 1. Total Transactions
total_transactions = transactions['transaction_id'].nunique()

# 2. Total Transaction Amount
total_transaction_amount = transactions['transaction_amount'].sum()

# 3. Total Fraud Transactions
fraud_transactions = transactions['is_fraud'].sum()

# 4. Fraud Rate (%)
fraud_rate = (
    fraud_transactions / total_transactions
) * 100

# 5. Fraud Transaction Amount
fraud_transaction_amount = transactions.loc[
    transactions['is_fraud'] == 1,
    'transaction_amount'
].sum()

# 6. Fraud Loss Rate (%)
fraud_loss_rate = (
    fraud_transaction_amount /
    total_transaction_amount
```

```

) * 100

# 7. Average Fraud Transaction Amount
average_fraud_amount = transactions.loc[
    transactions['is_fraud'] == 1,
    'transaction_amount'
].mean()

# 8. Total Accounts
total_accounts = account_profiles['account_id'].nunique()

# 9. High-Risk Accounts
high_risk_accounts = account_profiles.loc[
    account_profiles['is_high_risk'] == 1,
    'account_id'
].nunique()

# 10. High-Risk Account Rate (%)
high_risk_account_rate = (
    high_risk_accounts / total_accounts
) * 100

# 11. Fraudster Accounts
fraudster_accounts = account_profiles.loc[
    account_profiles['is_fraudster'] == 1,
    'account_id'
].nunique()

# 12. Fraudster Account Rate (%)
fraudster_account_rate = (
    fraudster_accounts / total_accounts
) * 100

# 13. Average Account Risk Score
average_account_risk_score = (
    account_profiles['account_risk_score'].mean()
)

# 14. Foreign Transaction Fraud Rate (%)
foreign_transactions = transactions[
    transactions['is_foreign_transaction'] == 1
]

foreign_fraud_rate = (
    foreign_transactions['is_fraud'].mean()
) * 100

# 15. No-2FA Fraud Rate (%)
no_2fa_transactions = transactions[
    transactions['has_2fa'] == 0
]

no_2fa_fraud_rate = (
    no_2fa_transactions['is_fraud'].mean()
) * 100

# 16. Unknown Device Fraud Rate (%)
unknown_device_transactions = transactions[
    transactions['device_known'] == 0
]

```

```

unknown_device_fraud_rate = (
    unknown_device_transactions['is_fraud'].mean()
) * 100

# 17. Night-Time Fraud Rate (%)
night_transactions = transactions[
    transactions['transaction_hour'].between(0, 5)
]

night_fraud_rate = (
    night_transactions['is_fraud'].mean()
) * 100

# 18. Average Fraud IP Risk
average_fraud_ip_risk = transactions.loc[
    transactions['is_fraud'] == 1,
    'ip_risk_score'
].mean()

# 19. Fraud Amount Ratio
overall_average_amount = transactions[
    'transaction_amount'
].mean()

fraud_amount_ratio = (
    average_fraud_amount /
    overall_average_amount
)

# 20. Average Fraud Transaction Velocity
average_fraud_velocity = transactions.loc[
    transactions['is_fraud'] == 1,
    'transactions_last_1h'
].mean()

kpis = {
    'Total Transactions': total_transactions,
    'Total Transaction Amount': total_transaction_amount,
    'Fraud Transactions': fraud_transactions,
    'Fraud Rate (%)': fraud_rate,
    'Fraud Transaction Amount': fraud_transaction_amount,
    'Fraud Loss Rate (%)': fraud_loss_rate,
    'Average Fraud Transaction Amount': average_fraud_amount,
    'Total Accounts': total_accounts,
    'High-Risk Accounts': high_risk_accounts,
    'High-Risk Account Rate (%)': high_risk_account_rate,
    'Fraudster Accounts': fraudster_accounts,
    'Fraudster Account Rate (%)': fraudster_account_rate,
    'Average Account Risk Score': average_account_risk_score,
    'Foreign Transaction Fraud Rate (%)': foreign_fraud_rate,
    'No-2FA Fraud Rate (%)': no_2fa_fraud_rate,
    'Unknown Device Fraud Rate (%)': unknown_device_fraud_rate,
    'Night-Time Fraud Rate (%)': night_fraud_rate,
    'Average Fraud IP Risk': average_fraud_ip_risk,
    'Fraud Amount Ratio': fraud_amount_ratio,
    'Average Fraud Transaction Velocity': average_fraud_velocity
}

```



```

for k, v in kpis.items():
    if isinstance(v, float):
        print(f"{k}: {v:,.2f}")
    else:
        print(f"{k}: {v:,}")

```

Total Transactions: 1,000,000
 Total Transaction Amount: 183,738,263.66
 Fraud Transactions: 17,143
 Fraud Rate (%): 1.71
 Fraud Transaction Amount: 12,516,734.09
 Fraud Loss Rate (%): 6.81
 Average Fraud Transaction Amount: 730.14
 Total Accounts: 50,000
 High-Risk Accounts: 18
 High-Risk Account Rate (%): 0.04
 Fraudster Accounts: 13,336
 Fraudster Account Rate (%): 26.67
 Average Account Risk Score: 19.91
 Foreign Transaction Fraud Rate (%): 3.40
 No-2FA Fraud Rate (%): 2.03
 Unknown Device Fraud Rate (%): 8.72
 Night-Time Fraud Rate (%): 2.79
 Average Fraud IP Risk: 55.35
 Fraud Amount Ratio: 3.97
 Average Fraud Transaction Velocity: 3.97

Connect Python to PostgreSQL

```

In [21]: from sqlalchemy import create_engine
import pandas as pd

# Database connection details
username = "postgres"
password = "password"
host = "localhost"
port = "5432"
database = "fraud_db"

# Create connection
engine = create_engine(
    f"postgresql+psycopg2://postgres:password@localhost:5432/fraud_db"
)

# Test connection
try:
    with engine.connect() as connection:
        print("Connected to PostgreSQL successfully!")
except Exception as e:
    print("Connection failed:", e)

```

Connected to PostgreSQL successfully!

```

In [22]: transactions.to_sql(
    "transactions",
    engine,
    if_exists="replace",
    index=False
)

```

```

account_profiles.to_sql(
    "account_profiles",
    engine,
    if_exists="replace",
    index=False
)

fraud_patterns.to_sql(
    "fraud_patterns",
    engine,
    if_exists="replace",
    index=False
)

network_edges.to_sql(
    "network_edges",
    engine,
    if_exists="replace",
    index=False
)

time_series_stats.to_sql(
    "time_series_stats",
    engine,
    if_exists="replace",
    index=False
)

print("All tables loaded successfully!")

```

All tables loaded successfully!

```

In [23]: tables = pd.read_sql("""
SELECT table_schema, table_name
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;
""", engine)

tables

```

```

Out[23]:


|   | table_schema | table_name        |
|---|--------------|-------------------|
| 0 | public       | account_profiles  |
| 1 | public       | fraud_patterns    |
| 2 | public       | network_edges     |
| 3 | public       | time_series_stats |
| 4 | public       | transactions      |


```

Business Problems

1. Overall Transaction Summary

```
In [24]: pd.read_sql("""
SELECT
    COUNT(transaction_id) AS total_transactions,
    SUM(transaction_amount) AS total_transaction_amount,
    AVG(transaction_amount) AS average_transaction_amount,
    SUM(is_fraud) AS fraud_transactions
FROM transactions;
""",engine)
```

```
Out[24]:
```

	total_transactions	total_transaction_amount	average_transaction_amount	fraud_transactions
0	1000000	1.837383e+08	183.738264	17143

2. Overall Fraud Rate

```
In [25]: pd.read_sql("""SELECT
COUNT(transaction_id) AS total_transactions,
SUM(is_fraud) AS fraud_transactions,
ROUND(
    SUM(is_fraud) * 100.0 / COUNT(transaction_id),
    2
) AS fraud_rate
FROM transactions;
""",engine)
```

```
Out[25]:
```

	total_transactions	fraud_transactions	fraud_rate
0	1000000	17143.0	1.71

3. Fraud Amount and Fraud Loss Rate

```
In [26]: pd.read_sql("""
SELECT
    SUM(
        CASE
            WHEN is_fraud = 1 THEN transaction_amount
            ELSE 0
        END
    ) AS fraud_transaction_amount,

    SUM(transaction_amount) AS total_transaction_amount,

    ROUND(
        (
            SUM(
                CASE
                    WHEN is_fraud = 1 THEN transaction_amount
                    ELSE 0
                END
            ) * 100.0
        / NULLIF(SUM(transaction_amount), 0)
    )::numeric,
```

```

        2
    ) AS fraud_loss_rate

FROM transactions;
""", engine)

```

Out[26]:

	fraud_transaction_amount	total_transaction_amount	fraud_loss_rate
0	12516734.09	1.837383e+08	6.81

4. Fraud by Merchant Category

In [27]:

```

pd.read_sql("""
SELECT
    merchant_category,
    COUNT(transaction_id) AS total_transactions,
    SUM(is_fraud) AS fraud_transactions,

    ROUND(
        SUM(is_fraud) * 100.0
        / COUNT(transaction_id)::numeric,
        2
    ) AS fraud_rate,

    SUM(
        CASE
            WHEN is_fraud = 1 THEN transaction_amount
            ELSE 0
        END
    ) AS fraud_amount

FROM transactions

GROUP BY merchant_category

ORDER BY fraud_rate DESC;
""", engine)

```

Out[27]:

	merchant_category	total_transactions	fraud_transactions	fraud_rate	fraud_amount
0	crypto	19976	1067.0	5.34	2973178.82
1	money_transfer	30079	1242.0	4.13	2526927.54
2	gambling	9920	354.0	3.57	495852.72
3	travel	49367	1288.0	2.61	2076275.03
4	electronics	60229	1372.0	2.28	1554183.59
5	hotel	49490	734.0	1.48	489458.36
6	restaurant	120406	1771.0	1.47	199114.32
7	atm	79595	1163.0	1.46	695372.46
8	gas_station	80030	1157.0	1.45	176796.38
9	clothing	70072	1000.0	1.43	226042.00
10	online_retail	150118	2121.0	1.41	533229.23
11	utilities	40256	564.0	1.40	148102.96
12	grocery	180167	2481.0	1.38	324946.60
13	pharmacy	60295	829.0	1.37	97254.08



5. Top 10 Countries by Fraud Rate

```
In [28]: pd.read_sql("""
SELECT
    merchant_country,
    COUNT(transaction_id) AS total_transactions,
    SUM(is_fraud) AS fraud_transactions,

    ROUND(
        SUM(is_fraud) * 100.0
        / COUNT(transaction_id),
        2
    ) AS fraud_rate

FROM transactions

GROUP BY merchant_country

HAVING COUNT(transaction_id) >= 1000

ORDER BY fraud_rate DESC

LIMIT 10;
""",engine)
```

Out[28]:

	merchant_country	total_transactions	fraud_transactions	fraud_rate
0	CA	39392	1401.0	3.56
1	MX	19613	693.0	3.53
2	RO	19876	685.0	3.45
3	CN	19657	670.0	3.41
4	GB	39261	1329.0	3.39
5	FR	39239	1317.0	3.36
6	AU	38890	1300.0	3.34
7	NG	19749	659.0	3.34
8	BR	19550	647.0	3.31
9	DE	39190	1297.0	3.31

6. Foreign vs Domestic Fraud

```
In [29]: pd.read_sql("""
SELECT
    CASE
        WHEN is_foreign_transaction = 1
            THEN 'Foreign'
        ELSE 'Domestic'
    END AS transaction_type,

    COUNT(transaction_id) AS total_transactions,

    SUM(is_fraud) AS fraud_transactions,

    ROUND(
        SUM(is_fraud) * 100.0
        / COUNT(transaction_id),
        2
    ) AS fraud_rate,

    SUM(
        CASE
            WHEN is_fraud = 1
                THEN transaction_amount
            ELSE 0
        END
    ) AS fraud_amount

FROM transactions

GROUP BY
    CASE
        WHEN is_foreign_transaction = 1
            THEN 'Foreign'
        ELSE 'Domestic'
    END
""")
```

```
ORDER BY fraud_rate DESC;

""",engine)
```

Out[29]:

	transaction_type	total_transactions	fraud_transactions	fraud_rate	fraud_amount
0	Foreign	294417	9998.0	3.40	7236325.74
1	Domestic	705583	7145.0	1.01	5280408.35

7. 2FA Impact on Fraud

In [30]:

```
pd.read_sql("""
SELECT
    CASE
        WHEN has_2fa = 1 THEN '2FA Enabled'
        ELSE '2FA Disabled'
    END AS two_fa_status,

    COUNT(transaction_id) AS total_transactions,

    SUM(is_fraud) AS fraud_transactions,

    ROUND(
        SUM(is_fraud) * 100.0
        / COUNT(transaction_id),
        2
    ) AS fraud_rate

FROM transactions

GROUP BY
    CASE
        WHEN has_2fa = 1 THEN '2FA Enabled'
        ELSE '2FA Disabled'
    END

ORDER BY fraud_rate DESC;
""",engine)
```

Out[30]:

	two_fa_status	total_transactions	fraud_transactions	fraud_rate
0	2FA Disabled	352433	7160.0	2.03
1	2FA Enabled	647567	9983.0	1.54

8. Known vs Unknown Device

In [31]:

```
pd.read_sql("""
SELECT
    CASE
        WHEN device_known = 1
            THEN 'Known Device'
        ELSE 'Unknown Device'
```

```

END AS device_status,

COUNT(transaction_id) AS total_transactions,

SUM(is_fraud) AS fraud_transactions,

ROUND(
    SUM(is_fraud) * 100.0
    / COUNT(transaction_id),
    2
) AS fraud_rate,

ROUND(AVG(transaction_amount)::numeric, 2)
    AS average_transaction_amount

FROM transactions

GROUP BY
CASE
    WHEN device_known = 1
        THEN 'Known Device'
    ELSE 'Unknown Device'
END

ORDER BY fraud_rate DESC;
""",engine)

```

Out[31]:

	device_status	total_transactions	fraud_transactions	fraud_rate	average_transaction_a
--	---------------	--------------------	--------------------	------------	-----------------------

0	Unknown Device	107663	9393.0	8.72	
1	Known Device	892337	7750.0	0.87	



9. Fraud by Transaction Hour

In [32]:

```

pd.read_sql("""
SELECT
    transaction_hour,

COUNT(transaction_id) AS total_transactions,

SUM(is_fraud) AS fraud_transactions,

ROUND(
    SUM(is_fraud) * 100.0
    / COUNT(transaction_id),
    2
) AS fraud_rate

FROM transactions

GROUP BY transaction_hour

ORDER BY fraud_rate DESC;
""",engine)

```


Out[32]:

	transaction_hour	total_transactions	fraud_transactions	fraud_rate
0	2	41845	1224.0	2.93
1	4	42249	1188.0	2.81
2	0	41328	1154.0	2.79
3	3	41980	1173.0	2.79
4	1	41495	1125.0	2.71
5	5	41593	1117.0	2.69
6	23	41531	811.0	1.95
7	22	41785	804.0	1.92
8	14	41673	571.0	1.37
9	16	41819	560.0	1.34
10	15	41264	554.0	1.34
11	9	41762	556.0	1.33
12	12	41441	551.0	1.33
13	6	41681	549.0	1.32
14	11	42111	543.0	1.29
15	8	41507	526.0	1.27
16	19	41690	529.0	1.27
17	18	41542	524.0	1.26
18	10	41571	521.0	1.25
19	17	41272	516.0	1.25
20	7	41651	516.0	1.24
21	21	41727	515.0	1.23
22	20	41618	512.0	1.23
23	13	41865	504.0	1.20

10. Weekend vs Weekday Fraud

In [33]:

```
pd.read_sql("""
SELECT
    CASE
        WHEN is_weekend = 1 THEN 'Weekend'
        ELSE 'Weekday'
    END AS day_type,

    COUNT(transaction_id) AS total_transactions,

    SUM(is_fraud) AS fraud_transactions,
```

```

ROUND(
    SUM(is_fraud) * 100.0
    / COUNT(transaction_id),
    2
) AS fraud_rate,

SUM(
    CASE
        WHEN is_fraud = 1
        THEN transaction_amount
        ELSE 0
    END
) AS fraud_amount

FROM transactions

GROUP BY
    CASE
        WHEN is_weekend = 1 THEN 'Weekend'
        ELSE 'Weekday'
    END;
""" ,engine)

```

Out[33]:

	day_type	total_transactions	fraud_transactions	fraud_rate	fraud_amount
0	Weekday	712848	12171.0	1.71	8868607.16
1	Weekend	287152	4972.0	1.73	3648126.93

11. Account Fraud Summary

In [34]:

```

pd.read_sql("""
SELECT
    account_id,

    COUNT(transaction_id) AS total_transactions,

    SUM(transaction_amount) AS total_transaction_amount,

    SUM(is_fraud) AS fraud_transactions,

    SUM(
        CASE
            WHEN is_fraud = 1
            THEN transaction_amount
            ELSE 0
        END
    ) AS fraud_amount,

    ROUND(
        SUM(is_fraud) * 100.0
        / COUNT(transaction_id),
        2
    ) AS fraud_rate

FROM transactions

```

```
GROUP BY account_id

ORDER BY fraud_amount DESC

LIMIT 20;
""",engine)
```

Out[34]:

	account_id	total_transactions	total_transaction_amount	fraud_transactions	fraud_
0	ACC0016655	40	32730.96	2.0	2
1	ACC0000286	27	28663.24	1.0	2
2	ACC0015479	52	35952.16	2.0	2
3	ACC0019782	16	23748.63	1.0	2
4	ACC0045771	19	20000.20	2.0	1
5	ACC0034305	8	19449.59	1.0	1
6	ACC0031303	49	22927.52	1.0	1
7	ACC0010680	10	15837.36	1.0	1
8	ACC0011952	27	18345.59	3.0	1
9	ACC0018754	34	19577.02	2.0	1
10	ACC0008607	71	24863.32	1.0	1
11	ACC0029908	17	15618.53	1.0	1
12	ACC0009579	16	14237.68	2.0	1
13	ACC0049560	32	18193.99	2.0	1
14	ACC0006926	13	12797.49	3.0	1
15	ACC0025008	29	15089.41	2.0	1
16	ACC0036283	84	25157.35	4.0	1
17	ACC0006902	26	14560.88	2.0	1
18	ACC0038113	25	21487.98	1.0	1
19	ACC0009551	30	15921.73	3.0	1



12. Fraudster Account Analysis

```
In [35]: pd.read_sql("""
SELECT
    account_id,
    account_type,
    account_risk_score,
    fraud_rate,
    fraud_transaction_amount,
    average_ip_risk
```

```

FROM account_profiles

WHERE is_fraudster = 1

ORDER BY fraud_transaction_amount DESC;

""",engine)

```

Out[35]:

	account_id	account_type	account_risk_score	fraud_rate	fraud_transaction_amo
0	ACC0016655	personal	6.4	0.050000	2530
1	ACC0000286	personal	10.9	0.037037	2383
2	ACC0015479	personal	32.7	0.038462	2283
3	ACC0019782	personal	23.9	0.062500	2211
4	ACC0045771	personal	14.0	0.105263	1642
...	...	...	...	...	...
13331	ACC0044272	personal	12.1	0.076923	...
13332	ACC0011991	premium	11.6	0.035714	...
13333	ACC0040664	personal	23.5	0.055556	...
13334	ACC0025498	personal	15.7	0.017544	...
13335	ACC0015173	personal	39.9	0.062500	...

13336 rows × 6 columns



13. High-Risk vs Low-Risk Accounts

```

In [36]: pd.read_sql("""
SELECT
    CASE
        WHEN is_high_risk = 1
            THEN 'High Risk'
        ELSE 'Low Risk'
    END AS risk_category,

    COUNT(account_id) AS total_accounts,

    ROUND(AVG(fraud_rate)::numeric, 2)
        AS average_fraud_rate,

    ROUND(
        AVG(fraud_transaction_amount)::numeric,
        2
    ) AS average_fraud_amount

FROM account_profiles

GROUP BY

```

```

CASE
    WHEN is_high_risk = 1
        THEN 'High Risk'
    ELSE 'Low Risk'
END;
""" ,engine)

```

Out[36]:

	risk_category	total_accounts	average_fraud_rate	average_fraud_amount
0	Low Risk	49982	0.02	250.26
1	High Risk	18	0.07	460.14

14. Account Type Analysis

In [37]:

```

pd.read_sql("""
SELECT
    account_type,

    COUNT(account_id) AS total_accounts,

    ROUND(
        AVG(account_risk_score)::numeric,
        2
    ) AS average_risk_score,

    SUM(is_fraudster) AS fraudster_accounts,

    ROUND(
        SUM(is_fraudster)::numeric
        * 100
        / COUNT(account_id),
        2
    ) AS fraudster_rate,

    ROUND(
        AVG(fraud_rate)::numeric,
        2
    ) AS average_fraud_rate

FROM account_profiles

GROUP BY account_type

ORDER BY average_fraud_rate DESC;
""", engine)

```

Out[37]:

	account_type	total_accounts	average_risk_score	fraudster_accounts	fraudster_rate
0	personal	35061	19.90	9327.0	26.60
1	premium	4951	19.80	1278.0	25.81
2	business	9988	20.02	2731.0	27.34

15. High-Risk Fraud Accounts

```
In [38]: pd.read_sql("""
WITH overall AS (

    SELECT
        SUM(is_fraud) * 100.0
        / COUNT(transaction_id) AS overall_fraud_rate
    FROM transactions

),

account_summary AS (

    SELECT
        account_id,

        COUNT(transaction_id) AS total_transactions,

        SUM(is_fraud) AS fraud_transactions,

        SUM(
            CASE
                WHEN is_fraud = 1
                THEN transaction_amount
                ELSE 0
            END
        ) AS fraud_amount,

        SUM(is_fraud) * 100.0
        / COUNT(transaction_id) AS account_fraud_rate

    FROM transactions

    GROUP BY account_id

),

avg_fraud_amount AS (

    SELECT
        AVG(fraud_amount) AS average_fraud_amount

    FROM account_summary

)

SELECT
    a.account_id,
    a.total_transactions,
    a.fraud_transactions,
    a.fraud_amount,
    ROUND(a.account_fraud_rate, 2)
    AS account_fraud_rate

FROM account_summary a
```

```
CROSS JOIN overall o

CROSS JOIN avg_fraud_amount f

WHERE a.account_fraud_rate > o.overall_fraud_rate
      AND a.fraud_amount > f.average_fraud_amount

ORDER BY a.fraud_amount DESC;
""",engine)
```

Out[38]:

	account_id	total_transactions	fraud_transactions	fraud_amount	account_fraud_r
0	ACC0016655	40	2.0	25303.42	!
1	ACC0000286	27	1.0	23833.77	:
2	ACC0015479	52	2.0	22835.78	:
3	ACC0019782	16	1.0	22118.53	(
4	ACC0045771	19	2.0	16429.41	10
...	...	...	...	...	
7366	ACC0018462	11	1.0	251.13	9
7367	ACC0012947	11	1.0	250.99	9
7368	ACC0009008	51	1.0	250.94	.
7369	ACC0022671	37	1.0	250.84	2
7370	ACC0044746	29	1.0	250.83	:

7371 rows × 5 columns



16. Unusually Large Transactions

```
In [39]: pd.read_sql("""
SELECT
    transaction_id,
    account_id,
    transaction_amount,
    amount_vs_average_ratio,
    ip_risk_score,
    transactions_last_1h,
    is_fraud

FROM transactions

WHERE amount_vs_average_ratio > 3

ORDER BY amount_vs_average_ratio DESC;
""",engine)
```

Out[39]:

	transaction_id	account_id	transaction_amount	amount_vs_average_ratio	ip_r
0	TXN000151560	ACC0016655	25000.00	945.5371	
1	TXN000252519	ACC0029908	12878.39	589.9400	
2	TXN000380920	ACC0025008	6556.41	515.4410	
3	TXN000492986	ACC0047301	5747.60	498.0589	
4	TXN000465743	ACC0008668	9371.83	469.2954	
...	...	...	...	...	...
261283	TXN000812530	ACC0002727	222.44	3.0003	
261284	TXN000751659	ACC0004315	137.47	3.0002	
261285	TXN000902780	ACC0028983	136.84	3.0002	
261286	TXN000637168	ACC0006793	215.62	3.0001	
261287	TXN000925562	ACC0018717	473.06	3.0001	

261288 rows × 7 columns



17. High-Velocity Transactions

In [40]:

```
pd.read_sql("""
SELECT
    AVG(transactions_last_1h) AS average_velocity
FROM transactions;
""",engine)
```

Out[40]:

	average_velocity
0	1.050421

In [41]:

```
pd.read_sql("""
WITH avg_velocity AS (

    SELECT
        AVG(transactions_last_1h)
        AS overall_average_velocity

    FROM transactions

)

SELECT
    CASE
        WHEN t.transactions_last_1h >
            a.overall_average_velocity
        THEN 'High Velocity'
        ELSE 'Normal Velocity'
    END AS velocity_category,
```



```

COUNT(t.transaction_id)
    AS total_transactions,

SUM(t.is_fraud)
    AS fraud_transactions,

ROUND(
    SUM(t.is_fraud) * 100.0
    / COUNT(t.transaction_id),
    2
) AS fraud_rate

FROM transactions t

CROSS JOIN avg_velocity a

GROUP BY
CASE
    WHEN t.transactions_last_1h >
        a.overall_average_velocity
    THEN 'High Velocity'
    ELSE 'Normal Velocity'
END;
""" ,engine)

```

Out[41]:

	velocity_category	total_transactions	fraud_transactions	fraud_rate
0	High Velocity	275368	15513.0	5.63
1	Normal Velocity	724632	1630.0	0.22

18. Fraud Pattern Ranking

```

In [42]: pd.read_sql("""
SELECT
    fraud_pattern,
    fraud_transaction_count,
    fraud_share_pct,
    average_transaction_amount,
    average_ip_risk,
    no_2fa_pct,

    RANK() OVER (
        ORDER BY fraud_share_pct DESC
    ) AS fraud_pattern_rank

FROM fraud_patterns

ORDER BY fraud_pattern_rank;

""",engine)

```

Out[42]:

	fraud_pattern	fraud_transaction_count	fraud_share_pct	average_transaction_amo
0	card_not_present	5982	34.89	777
1	account_takeover	3432	20.02	1203
2	card_present_stolen	3120	18.20	550
3	friendly_fraud	1726	10.07	352
4	atm_fraud	1216	7.09	460
5	money_laundering	1011	5.90	93
6	identity_theft	656	3.83	1163



19. Monthly Fraud Trend & MOM Change

In [43]:

```
pd.read_sql("""
WITH monthly_fraud AS (

    SELECT
        DATE_TRUNC(
            'month',
            transaction_datetime
        ) AS month,

        COUNT(transaction_id)
        AS total_transactions,

        SUM(transaction_amount)
        AS total_transaction_amount,

        SUM(is_fraud)
        AS fraud_transactions,

        SUM(
            CASE
                WHEN is_fraud = 1
                THEN transaction_amount
                ELSE 0
            END
        ) AS fraud_amount,

        SUM(is_fraud) * 100.0
        / COUNT(transaction_id)
        AS fraud_rate

    FROM transactions

    GROUP BY
        DATE_TRUNC(
            'month',
            transaction_datetime
        )

)
```

```
SELECT
    month,
    total_transactions,
    total_transaction_amount,
    fraud_transactions,
    fraud_amount,
    ROUND(fraud_rate, 2) AS fraud_rate,

    ROUND(
        fraud_rate -
        LAG(fraud_rate) OVER (
            ORDER BY month
        ),
        2
    ) AS mom_change_in_fraud_rate

FROM monthly_fraud

ORDER BY month;
""",engine)
```

Out[43]:

	month	total_transactions	total_transaction_amount	fraud_transactions	fraud_amou
0	2022-01-01	28299	5188452.27	515.0	366173.
1	2022-02-01	25542	4577719.58	464.0	299601.
2	2022-03-01	28291	5281391.19	474.0	369268.
3	2022-04-01	27340	5051068.86	492.0	344193.
4	2022-05-01	28315	5324028.95	482.0	389085.
5	2022-06-01	27403	5070538.55	504.0	374615.
6	2022-07-01	28387	5126866.18	434.0	324964.
7	2022-08-01	28533	5284828.03	497.0	417542.
8	2022-09-01	27631	5018952.00	486.0	346662.
9	2022-10-01	28250	5210008.61	473.0	354941.
10	2022-11-01	27459	5034722.57	476.0	362540.
11	2022-12-01	28273	5300895.89	464.0	368638.
12	2023-01-01	28165	5111441.13	512.0	339111.
13	2023-02-01	25643	4687827.06	427.0	282003.
14	2023-03-01	28168	5179606.39	495.0	379669.
15	2023-04-01	27463	5034704.24	469.0	327309.
16	2023-05-01	28156	5191790.67	446.0	326335.
17	2023-06-01	27536	5016140.42	439.0	354935.
18	2023-07-01	28152	5209222.12	443.0	339856.
19	2023-08-01	28289	5269173.41	493.0	390201.
20	2023-09-01	27178	4915967.81	493.0	315980.

	month	total_transactions	total_transaction_amount	fraud_transactions	fraud_amou
21	2023-10-01	27982	5137822.30	456.0	317889.
22	2023-11-01	27360	5028125.10	494.0	334032.
23	2023-12-01	28270	5164995.19	490.0	365063.
24	2024-01-01	28196	5113414.22	470.0	361487.
25	2024-02-01	26595	4888355.00	531.0	383317.
26	2024-03-01	28348	5136240.92	450.0	318567.
27	2024-04-01	27442	5013979.53	454.0	279869.
28	2024-05-01	28365	5274320.86	510.0	358345.
29	2024-06-01	27467	5032716.44	483.0	327233.
30	2024-07-01	28434	5274195.33	498.0	394687.
31	2024-08-01	28411	5202654.79	501.0	340841.
32	2024-09-01	27521	5040713.49	466.0	330473.
33	2024-10-01	28248	5160299.26	440.0	324095.
34	2024-11-01	27410	5079564.49	451.0	363358.
35	2024-12-01	27478	5105520.81	471.0	343843.

20. Fraud Ring Analysis

```
In [44]: pd.read_sql("""
SELECT
    fraud_ring_id,

    COUNT(*) AS total_connections,

    COUNT(
        DISTINCT connection_type
    ) AS connection_types,
```

```

COUNT(
    DISTINCT account_a
) +
COUNT(
    DISTINCT account_b
) AS connected_account_count,

SUM(
    CASE
        WHEN both_accounts_fraud = 1
        THEN 1
        ELSE 0
    END
) AS fraud_to_fraud_connections

FROM network_edges

WHERE fraud_ring_id IS NOT NULL

GROUP BY fraud_ring_id

ORDER BY fraud_to_fraud_connections DESC,
         total_connections DESC;
""",engine)

```

Out[44]:

	fraud_ring_id	total_connections	connection_types	connected_account_count	fraud
0	RING0072	55	4	20	
1	RING0088	55	4	20	
2	RING0084	55	4	20	
3	RING0182	55	4	20	
4	RING0179	55	4	20	
...	...	...	...	...	...
196	RING0153	3	3	4	
197	RING0116	3	3	4	
198	RING0151	3	2	4	
199	RING0070	3	3	4	
200	No Ring	3000	3	5825	

201 rows × 5 columns



Conclusion

The analysis shows that fraud is relatively infrequent in transaction volume but has a disproportionately large financial impact. The overall fraud rate is **1.71%**, while fraudulent transactions account for **6.81% of total transaction value**, highlighting the importance of focusing on potential financial loss rather than transaction count alone.

The strongest behavioral indicators identified were **unknown devices and high transaction velocity**, with fraud rates of **8.72% and 5.63% respectively**, compared with much lower rates for known devices and normal transaction velocity. Foreign transactions, disabled 2FA, unusual transaction hours, and high-risk merchant categories also showed elevated fraud rates.

Fraud is concentrated in a small number of major patterns, with **card-not-present, account takeover, and stolen-card fraud accounting for approximately 73% of fraudulent transactions**. The network analysis also indicates that fraudulent activity can occur through connected groups of accounts, suggesting that transaction-level monitoring alone may not be sufficient.

Overall, the analysis supports a **multi-layered fraud detection strategy** combining transaction behavior, account risk, authentication signals, merchant risk, temporal patterns, and network relationships. This approach can help financial institutions identify suspicious activity earlier, prioritize investigations, and reduce potential fraud losses.

In []: